

# **Complete Data Structures & Algorithms Syllabus**

## **Module 1: Arrays & Strings**

### **1.1 Array Basics**

basics, declaration and initialization, memory layout (contiguous allocation), indexing (0-based vs 1-based), traversal patterns, input/output operations, array vs list distinctions

### **1.2 Searching**

linear search basics, find element in array, find min/max element, count occurrences, binary search (sorted arrays), search in rotated sorted array, find first/last occurrence, peak element finding, search in 2D matrix

### **1.3 Two Pointers Technique**

opposite direction pointers, two sum in sorted array, pair with given sum, remove duplicates from sorted array, container with most water, valid palindrome check, move zeros to end, sort colors (Dutch National Flag)

### **1.4 Sliding Window**

fixed size window problems, maximum sum subarray of size k, first negative in every window, count occurrences of anagrams, variable size windows, longest substring without repeating characters, minimum window substring, longest substring with k distinct characters, minimum size subarray sum

### **1.5 Prefix Sum & Difference Arrays**

prefix sum array construction, range sum queries, subarray sum equals k, maximum subarray sum (Kadane's algorithm), equilibrium index, difference array for range updates, maximum sum with exactly k elements

### **1.6 Sorting Algorithms**

bubble sort mechanism, selection sort approach, insertion sort for nearly sorted, merge sort divide-and-conquer, quick sort partitioning, counting sort for limited range, radix sort for integers, stability in sorting, comparison vs non-comparison sorts

### **1.7 Array Manipulation Advanced**

rotate array (left/right), leaders in array, stock buy-sell problems (single/multiple transactions), rain water trapping, next greater element, product of array except self, missing number in sequence, duplicate detection techniques

### **1.8 String Fundamentals**

string traversal and manipulation, reverse string/words, check anagrams, longest common prefix, string matching (naive), pattern searching basics, palindrome variations, character frequency counting

### **1.9 String Pattern Matching**

KMP algorithm (pattern preprocessing), Rabin-Karp (rolling hash), Z-algorithm applications, longest palindromic substring, string compression problems, word break problem

### **1.10 Matrix/2D Array Problems**

spiral matrix traversal, rotate matrix 90 degrees, set matrix zeros, search in row-column sorted matrix, diagonal traversal, boundary traversal, maximum rectangle in matrix, word search in grid

---

## Module 2: Linked Lists

### 2.1 Singly Linked List Basics

node structure (data + next pointer), insertion (beginning/end/position), deletion operations, traversal and display, length calculation, memory allocation dynamics

### 2.2 Singly Linked List Problems

reverse linked list (iterative/recursive), detect cycle (Floyd's algorithm), find middle element (slow-fast pointers), remove nth node from end, merge two sorted lists, intersection point of two lists, delete node with only pointer to it

### 2.3 Doubly Linked List

node structure (prev + data + next), insertion/deletion operations, advantages over singly linked, reverse doubly linked list, pairs with given sum in DLL, remove duplicates from sorted DLL

### 2.4 Circular Linked Lists

circular singly linked list basics, circular doubly linked list, detecting circularity, josephus problem, splitting circular list

### 2.5 Advanced Linked List Problems

merge k sorted lists, rotate list by k positions, add two numbers represented as lists, clone list with random pointer, flatten multilevel linked list, LRU cache implementation using list

---

## Module 3: Stacks

### 3.1 Stack Basics

LIFO principle, array implementation, linked list implementation, push/pop/peek operations, isEmpty and isFull checks, applications overview

### 3.2 Stack Expression Problems

balanced parentheses checking, infix to postfix conversion, infix to prefix conversion, postfix evaluation, prefix evaluation, redundant brackets detection

### 3.3 Stack Monotonic Problems

next greater element (circular/non-circular), next smaller element, previous greater/smaller element, largest rectangle in histogram, maximal rectangle in binary matrix, stock span problem

### 3.4 Stack Advanced

implement queue using stacks, min stack ( $O(1)$  getMin), sort stack using recursion, celebrity problem, decode string with nested brackets, simplify path (directory navigation)

---

## Module 4: Queues

## 4.1 Queue Basics

FIFO principle, array implementation (circular queue), linked list implementation, enqueue/dequeue operations, front/rear pointer management

## 4.2 Deque (Double-Ended Queue)

operations at both ends, sliding window maximum using deque, maximum of all subarrays of size k, implementing stack using deque

## 4.3 Priority Queue (Heap)

min heap vs max heap properties, heap implementation (array-based), heapify operation, insertion and deletion, extract min/max, kth largest/smallest element, merge k sorted arrays, top k frequent elements

## 4.4 Queue Advanced Problems

implement stack using queues, first non-repeating character in stream, generate binary numbers from 1 to n, level order traversal using queue, rotting oranges (BFS problem)

---

## Module 5: Hashing

### 5.1 Hash Table Basics

hash function concepts, collision handling (chaining vs open addressing), load factor, rehashing, time complexity analysis

### 5.2 Hash Map/Set Problems

two sum problem, group anagrams, longest consecutive sequence, subarray with sum zero, count pairs with given sum, first repeating element, check if array is subset of another

### 5.3 Frequency Counting

count distinct elements, frequency of array elements, elements with frequency k, find duplicates, uncommon words from two sentences, top k frequent words

### 5.4 Hashing Advanced

longest subarray with sum k, longest subarray with equal 0s and 1s, count subarrays with given XOR, check if two arrays are equal (same frequency), four sum problem using hashing

---

## Module 6: Trees (Binary Trees)

### 6.1 Binary Tree Basics

tree terminology (root, leaf, parent, child, height, depth), binary tree properties, full/complete/perfect binary tree distinctions, array vs linked representation

### 6.2 Tree Traversals

preorder traversal (recursive/iterative), inorder traversal (recursive/iterative), postorder traversal (recursive/iterative), level order traversal (BFS), zigzag level order, vertical order traversal, boundary traversal, diagonal traversal

### 6.3 Binary Tree Construction

construct tree from inorder+preorder, construct tree from inorder+postorder, construct tree from level order, serialize and deserialize binary tree

### 6.4 Binary Tree Properties

height/depth calculation, diameter of binary tree, check balanced tree, check if trees are identical, symmetric tree check, count nodes in complete tree, check if subtree of another tree

### 6.5 Binary Tree Path Problems

root to leaf paths, sum root to leaf numbers, path sum (target sum exists), path sum II (all paths), binary tree maximum path sum, longest zigzag path, all nodes at distance k

### 6.6 Lowest Common Ancestor (LCA)

LCA in binary tree, LCA with parent pointers, LCA of deepest leaves, distance between two nodes using LCA, kth ancestor of node (binary lifting concept)

### 6.7 Binary Tree Advanced

flatten binary tree to linked list, convert binary tree to doubly linked list, maximum width of binary tree, populating next right pointers, binary tree camera placement, distribute coins in binary tree

---

## Module 7: Binary Search Trees (BST)

### 7.1 BST Basics

BST property (left < root < right), search in BST, insertion in BST, deletion in BST (3 cases), inorder gives sorted sequence

### 7.2 BST Construction & Conversion

construct BST from preorder, construct BST from sorted array, convert sorted list to BST, BST to greater sum tree, inorder successor/predecessor

### 7.3 BST Validation & Properties

validate BST, kth smallest/largest element in BST, count BST nodes in range, check if preorder is valid BST, unique BSTs count (Catalan numbers)

### 7.4 BST Advanced

two sum in BST, LCA in BST, BST iterator, recover BST (two nodes swapped), merge two BSTs, pairs from two BSTs with given sum

---

## Module 8: Advanced Trees

## 8.1 AVL Trees

balance factor concept, rotations (LL, RR, LR, RL), insertion with balancing, deletion with balancing, height-balanced property maintenance

## 8.2 Red-Black Trees

properties (color rules), insertion with recoloring/rotation, deletion operations, self-balancing mechanism, comparison with AVL

## 8.3 Segment Trees

range query problems, lazy propagation, point updates, range updates, range minimum query, range sum query, range GCD query

## 8.4 Binary Indexed Tree (Fenwick Tree)

prefix sum with updates, point update range query, range update point query, 2D BIT for matrix queries

## 8.5 Trie (Prefix Tree)

trie structure (character nodes), insertion and search, word search (prefix/complete word), autocomplete implementation, longest common prefix using trie, word break using trie, implement phone directory

## 8.6 Suffix Arrays & Trees

suffix array construction, LCP array, pattern matching using suffix array, suffix tree applications, longest repeated substring

---

## Module 9: Heaps & Priority Queues

### 9.1 Min Heap & Max Heap

heap property maintenance, build heap from array, heap sort algorithm, kth largest/smallest in stream

### 9.2 Heap Advanced Problems

merge k sorted lists/arrays using heap, connect n ropes with minimum cost, reorganize string (no adjacent duplicates), task scheduler with cooling period, find median in data stream (two heaps), sliding window median

### 9.3 Special Heap Problems

k closest points to origin, top k frequent elements, kth largest element in array, IPO maximum capital problem

---

## Module 10: Graphs - Representation & Traversal

### 10.1 Graph Basics

directed vs undirected graphs, weighted vs unweighted, adjacency matrix representation, adjacency list representation, edge list representation, graph terminology (degree, path, cycle, connected components)

### 10.2 Depth-First Search (DFS)

DFS traversal (recursive/iterative), connected components count, path finding using DFS, detect cycle in undirected graph, detect cycle in directed graph, topological sort using DFS, time stamps (discovery/finish), bridge detection, articulation points

### **10.3 Breadth-First Search (BFS)**

BFS traversal using queue, shortest path in unweighted graph, level of each node, bipartite graph check, 0-1 BFS for special graphs, multi-source BFS, word ladder problem

### **10.4 Graph Coloring & Bipartite**

two-coloring using BFS/DFS, check if graph is bipartite, m-coloring problem, graph coloring variations

---

## **Module 11: Graphs - Shortest Paths**

### **11.1 Single Source Shortest Path**

Dijkstra's algorithm (priority queue), path reconstruction, shortest path in weighted DAG using topological sort, Bellman-Ford algorithm (negative weights), detect negative cycles

### **11.2 All Pairs Shortest Path**

Floyd-Warshall algorithm, transitive closure, detect negative cycles, diameter of graph

### **11.3 Shortest Path Variations**

path with minimum effort, cheapest flights within k stops, network delay time, path with maximum probability

---

## **Module 12: Graphs - Advanced Algorithms**

### **12.1 Minimum Spanning Tree (MST)**

Kruskal's algorithm (union-find), Prim's algorithm (priority queue), minimum cost to connect all points, critical connections in network

### **12.2 Union-Find (Disjoint Set Union)**

union by rank, path compression, find operation optimization, number of connected components, detect cycle using union-find, accounts merge problem, redundant connection

### **12.3 Strongly Connected Components**

Kosaraju's algorithm, Tarjan's algorithm, condensation graph, critical connections

### **12.4 Network Flow**

max flow min cut theorem, Ford-Fulkerson method, Edmonds-Karp algorithm, bipartite matching using flow

### **12.5 Advanced Graph Problems**

course schedule (topological sort), alien dictionary, cheapest flights with k stops, reconstruct itinerary, minimize malware spread, evaluate division (graph with weights)

---

## **Module 13: Dynamic Programming - Fundamentals**

### **13.1 DP Introduction**

overlapping subproblems concept, optimal substructure property, memoization (top-down), tabulation (bottom-up), space optimization techniques

### **13.2 Fibonacci Pattern**

fibonacci sequence, climbing stairs, house robber, decode ways, min cost climbing stairs, tiling problem variations

### **13.3 Kadane's Algorithm & Variations**

maximum subarray sum, maximum product subarray, maximum sum circular subarray, longest turbulent subarray, maximum alternating subsequence sum

### **13.4 Linear DP**

longest increasing subsequence (LIS), longest decreasing subsequence, number of LIS, russian doll envelopes, maximum sum increasing subsequence

---

## **Module 14: Dynamic Programming - Grid & Matrix**

### **14.1 Grid Pathfinding**

unique paths in grid, unique paths with obstacles, minimum path sum, maximum path sum, dungeon game, cherry pickup

### **14.2 Matrix DP**

maximal square in matrix, maximal rectangle, count square submatrices, largest plus sign, bomb enemy problem

---

## **Module 15: Dynamic Programming - Sequences**

### **15.1 Longest Common Subsequence (LCS)**

LCS basic problem, print LCS, longest common substring, shortest common supersequence, minimum insertions/deletions to convert string

### **15.2 Edit Distance Pattern**

edit distance (Levenshtein), minimum deletions to make palindrome, distinct subsequences, wildcard pattern matching, regular expression matching, interleaving strings

### **15.3 Palindrome DP**

longest palindromic subsequence, longest palindromic substring, palindrome partitioning (minimum cuts), count palindromic substrings, palindrome partitioning II

---

## **Module 16: Dynamic Programming - Knapsack Variants**

### **16.1 0/1 Knapsack**

basic knapsack problem, subset sum problem, partition equal subset sum, minimum subset sum difference, count subsets with given sum, target sum (assign +/-), last stone weight II

### **16.2 Unbounded Knapsack**

unbounded knapsack basic, coin change (minimum coins), coin change II (number of ways), rod cutting problem, perfect squares, minimum cost for tickets

### **16.3 Bounded Knapsack**

bounded knapsack with quantity limits, fractional knapsack (greedy approach for comparison)

---

## **Module 17: Dynamic Programming - Advanced Patterns**

### **17.1 Interval DP**

matrix chain multiplication, palindrome partitioning, burst balloons, minimum cost to merge stones, optimal binary search tree

### **17.2 Digit DP**

count numbers with unique digits, numbers at most n given digit set, numbers with repeated digits, count special integers

### **17.3 Bitmask DP**

traveling salesman problem, assignment problem, shortest path visiting all nodes, partition into disjoint intervals, minimum cost to connect cities

### **17.4 DP on Trees**

house robber III (tree DP), binary tree cameras, longest path in tree, tree diameter using DP, all possible full binary trees

### **17.5 State Machine DP**

best time to buy/sell stock (multiple variations), paint house series, paint fence, flip string to monotone increasing

### **17.6 Game Theory DP**

stone game, predict the winner, nim game, cat and mouse game, divisor game

---

## **Module 18: Greedy Algorithms**

### **18.1 Greedy Fundamentals**

greedy choice property, optimal substructure, activity selection problem, fractional knapsack, huffman coding basics



## **18.2 Greedy Interval Problems**

merge intervals, insert interval, non-overlapping intervals, meeting rooms, meeting rooms II, minimum arrows to burst balloons

## **18.3 Greedy Scheduling**

task scheduler, reorganize string, minimum platforms required, job sequencing problem

## **18.4 Greedy Advanced**

jump game (can reach end), jump game II (minimum jumps), gas station circular tour, candy distribution, boats to save people, two city scheduling

---

## **Module 19: Backtracking**

### **19.1 Backtracking Basics**

backtracking template, state space tree, pruning techniques, when to use backtracking

### **19.2 Permutations & Combinations**

generate permutations, generate combinations, combination sum (unique elements), combination sum II (with duplicates), combination sum III (fixed count)

### **19.3 Subset Generation**

generate all subsets, subsets with duplicates, generate parentheses, letter combinations of phone number, palindrome partitioning

### **19.4 Constraint Satisfaction**

N-queens problem, sudoku solver, word search in grid, word search II (trie optimization), rat in a maze, knight's tour

### **19.5 Backtracking Advanced**

restore IP addresses, expression add operators, remove invalid parentheses, beautiful arrangement, matchsticks to square

---

## **Module 20: Bit Manipulation**

### **20.1 Bitwise Basics**

AND, OR, XOR, NOT operations, left/right shift operators, check if power of two, count set bits (Kernighan's algorithm), toggle/set/clear specific bit

### **20.2 Bit Manipulation Problems**

single number (XOR properties), single number II (3 duplicates), single number III (two unique), missing number using XOR, power set using bitmask

### **20.3 Advanced Bit Problems**

reverse bits, hamming distance, total hamming distance, maximum XOR of two numbers, maximum AND value pair, bitwise AND of numbers range

---

## **Module 21: Mathematical Algorithms**

### **21.1 Number Theory**

prime numbers (Sieve of Eratosthenes), GCD (Euclidean algorithm), LCM calculation, modular arithmetic, fast exponentiation (binary exponentiation), prime factorization

### **21.2 Combinatorics**

factorial and permutations, combinations formula, Pascal's triangle, Catalan numbers, binomial coefficient, pigeonhole principle

### **21.3 Mathematical Problems**

integer to Roman and vice versa, power function implementation, square root using binary search, nth digit problem, happy number, ugly number series, super ugly numbers

---

## **Module 22: Advanced Techniques**

### **22.1 Divide and Conquer**

merge sort paradigm, quick sort paradigm, closest pair of points, Strassen's matrix multiplication, median of two sorted arrays, count inversions in array

### **22.2 Two Pointers Advanced**

3Sum problem, 4Sum problem, trapping rain water, container with most water, sort colors without extra space

### **22.3 Monotonic Stack/Queue**

largest rectangle in histogram, maximal rectangle, sliding window maximum, sum of subarray minimums, next greater element variations

### **22.4 Binary Search Advanced**

search in rotated sorted array, search insert position, find peak element, koko eating bananas, minimum in rotated sorted array, capacity to ship packages within d days, split array largest sum

### **22.5 Sweep Line Algorithm**

merge intervals using sweep line, skyline problem, meeting rooms II optimal solution, rectangle overlap problems

---

## **Module 23: String Algorithms Advanced**

### **23.1 Pattern Matching Advanced**

KMP algorithm implementation, Rabin-Karp for multiple patterns, Boyer-Moore algorithm, Aho-Corasick for multiple pattern search

### **23.2 String Hashing**

polynomial rolling hash, double hashing to avoid collisions, longest duplicate substring, shortest palindrome using hashing

### **23.3 Manacher's Algorithm**

longest palindromic substring in  $O(n)$ , finding all palindrome substrings efficiently

---

## **Module 24: Design & Implementation**

### **24.1 Data Structure Design**

LRU cache (hash map + doubly linked list), LFU cache, implement Trie, design HashMap, design HashSet, min stack, max stack

### **24.2 System Design for DSA**

design twitter feed, design search autocomplete, design file system, design in-memory database, design tic-tac-toe, design snake game

---

## **Module 25: Problem Solving Patterns**

### **25.1 Fast & Slow Pointers**

cycle detection (Floyd's), find duplicate number, linked list cycle II (entry point), happy number problem, palindrome linked list

### **25.2 Merge Intervals Pattern**

merge intervals, insert interval, interval list intersections, employee free time, minimum meeting rooms

### **25.3 Cyclic Sort**

find missing number, find all duplicates, find all missing numbers, first missing positive

### **25.4 Modified Binary Search**

ceiling of a number, floor of a number, next letter, find range, search bitonic array

### **25.5 Top K Elements**

kth largest element, k closest points, top k frequent elements, k closest numbers, frequency sort

### **25.6 K-way Merge**

merge k sorted lists, kth smallest in sorted matrix, smallest range covering k lists, find k pairs with smallest sums

---

## Tracking Progress Template

### How to use this syllabus:

1. Mark each subtopic as "Not Started", "In Progress", or "Completed"
2. For each subtopic, solve at least 3-5 problems covering the concepts
3. Revisit "In Progress" topics weekly
4. Build a personal problem tracker (spreadsheet/notion) with columns:
  - Module.Unit - Topic Name
  - Date Started
  - Problems Solved (with links)
  - Difficulty Level
  - Status
  - Notes/Patterns Learned

### Suggested Problem Sources:

- LeetCode, Codeforces, HackerRank, GeeksforGeeks
- Focus on understanding patterns rather than memorizing solutions
- Track time complexity and space complexity for each solution
- Revisit solved problems after 1 week, 1 month, 3 months

### Estimated Timeline:

- Beginner: 6-8 months (2-3 hours/day)
- Intermediate: 3-4 months (3-4 hours/day)
- Advanced: 1-2 months (revision and hard problems)